

Unit 3: Data Cleaning Scenario Solutions

Comprehensive Python & Pandas Solutions for Data Cleanup, Validation, and Standardisation

Scenario 1: Why Clean Data?

Problem: A raw dataset contains missing values and inconsistent data formats (e.g., text representations of numbers). We need to clean it before performing analysis.

PYTHON SOLUTION

```
import pandas as pd

# Sample raw data
data = {
    "Name": ["Alice", "Bob", None, "Charlie", "David"],
    "Age": [25, "Thirty", 29, None, 22],
    "Salary": [50000, 55000, 60000, 58000, "Sixty-Five Thousand"]
}
df = pd.DataFrame(data)

print("Raw Data:")
print(df)

# Handling missing values and standardizing data formats
df["Name"].fillna("Unknown", inplace=True)
df["Age"] = pd.to_numeric(df["Age"], errors="coerce") # Converts text 'Thirty' to NaN
df["Salary"] = df["Salary"].replace("Sixty-Five Thousand", 65000).astype(int)

print("Cleaned Data:")
print(df)
```

EXPECTED OUTPUT

```
Raw Data:
   Name  Age  Salary
0  Alice   25   50000
1   Bob  Thirty   55000
2   None   29   60000
3 Charlie  None   58000
4  David   22  Sixty-Five Thousand

Cleaned Data:
   Name  Age  Salary
0  Alice  25.0  50000
1   Bob   NaN  55000
2 Unknown  29.0  60000
3 Charlie  NaN  58000
4  David  22.0  65000
```

Explanation: Missing values in `Name` are imputed with `"Unknown"`. The `pd.to_numeric(..., errors="coerce")` function safely converts numeric strings to floats while coercing invalid string formats like `"Thirty"` into `NaN`. Custom string values in `Salary` are replaced and explicitly cast to integers.

Scenario 2: Identifying Values for Data Cleanup

Problem: Identify missing values and inspect the data types across all columns in a dataset to determine required cleaning operations.

PYTHON SOLUTION

```
# Checking for missing values count per column
print("Missing Values Count:")
print(df.isnull().sum())

# Inspecting data types of each column
print("Data Types:")
print(df.dtypes)
```

EXPECTED OUTPUT

```
Missing Values Count:
Name      0
Age       2
Salary    0
dtype: int64

Data Types:
Name      object
Age       float64
Salary    int64
dtype: object
```

Explanation: `df.isnull().sum()` summarizes missing values per column, highlighting that `Age` has 2 missing entries. `df.dtypes` verifies that columns are stored in appropriate data types (`'float64'`, `'int64'`, `'object'`).

Scenario 3: Formatting Data

Problem: The `Date` column contains multiple date formats (`'YYYY-MM-DD'`, `'DD-MM-YYYY'`, `'MM/DD/YYYY'`, text string). We need to standardize all entries.

PYTHON SOLUTION

```
import pandas as pd

data = {'Date': ['2025-01-01', '01-02-2025', '03/05/2025', 'April 7, 2025']}
df = pd.DataFrame(data)

# Converting all date formats to standard ISO format (YYYY-MM-DD)
df['Date'] = pd.to_datetime(df['Date'])
print(df)
```

EXPECTED OUTPUT

```
   Date
0 2025-01-01
1 2025-01-02
2 2025-03-05
3 2025-04-07
```

Explanation: Pandas' `pd.to_datetime()` parses non-uniform date representations seamlessly and coerces them into unified `datetime64[ns]` objects, displaying them in standard `YYYY-MM-DD` format.

Scenario 4: Finding Outliers and Bad Data

Problem: Detect outliers in numerical sales data using the Interquartile Range (IQR) method.

PYTHON SOLUTION

```
import pandas as pd
import numpy as np

data = {'Sales': [200, 250, 300, 1500, 260, 270, 280, 10000]}
df = pd.DataFrame(data)

# Calculate Q1 (25th percentile) and Q3 (75th percentile)
Q1 = np.percentile(df["Sales"], 25)
Q3 = np.percentile(df["Sales"], 75)
IQR = Q3 - Q1

# Define lower and upper bound thresholds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter outliers
outliers = df[(df["Sales"] < lower_bound) | (df["Sales"] > upper_bound)]
print("Outliers:")
print(outliers)
```

EXPECTED OUTPUT

```
Outliers:
  Sales
3  1500
7 10000
```

Explanation: The IQR method defines valid bounds between $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$. Values 1500 and 10000 fall outside this range and are correctly flagged as statistical outliers.

Scenario 5: Finding Duplicates

Problem: Identify duplicate records in a dataset and remove them to ensure data uniqueness.

PYTHON SOLUTION

```
import pandas as pd

data = {'ID': [1, 2, 3, 2, 4, 5, 3], 'Value': [100, 200, 300, 200, 400, 500, 300]}
df = pd.DataFrame(data)

# Find duplicates
duplicates = df[df.duplicated()]
print("Duplicates Found:")
print(duplicates)

# Remove duplicates
df_cleaned = df.drop_duplicates()
print("Cleaned Data:")
print(df_cleaned)
```

EXPECTED OUTPUT

Duplicates Found:

	ID	Value
3	2	200
6	3	300

Cleaned Data:

	ID	Value
0	1	100
1	2	200
2	3	300
4	4	400
5	5	500

Explanation: `df.duplicated()` returns a boolean mask indicating rows that are exact duplicates of earlier rows.
`df.drop_duplicates()` eliminates duplicate occurrences, retaining only original unique rows.

Scenario 6: Fuzzy Matching

Problem: Match similar but slightly misspelled strings (e.g., typos in names) against a target reference query.

PYTHON SOLUTION

```
from fuzzywuzzy import fuzz, process

names = ["John Doe", "Jon Doe", "Jane Doe", "J0hn Doe"]
query = "John Doe"

# Extract top 3 matches based on Levenshtein distance similarity scoring
matches = process.extract(query, names, limit=3)
print("Fuzzy Matches:")
print(matches)
```

EXPECTED OUTPUT

```
Fuzzy Matches:
[('John Doe', 100), ('Jon Doe', 95), ('J0hn Doe', 80)]
```

Explanation: `fuzzywuzzy.process.extract()` uses string similarity algorithms (Levenshtein distance) to rank target string candidate matches against a query with similarity confidence percentages (0-100%).

Scenario 7: RegEx Matching

Problem: Validate email strings using Regular Expressions (RegEx) to filter out malformed email addresses.

PYTHON SOLUTION

```
import re

emails = ["test@example.com", "user.name@domain.com", "invalid-email@", "hello@world"]

def is_valid_email(email):
    pattern = r'^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.$'
    return bool(re.match(pattern, email))

invalid_emails = [email for email in emails if not is_valid_email(email)]
print("Invalid Emails:", invalid_emails)
```

EXPECTED OUTPUT

```
Invalid Emails: ['invalid-email@', 'hello@world']
```

Explanation: The RegEx pattern checks for a valid username prefix, an `@` symbol, a valid domain name, and a top-level domain extension (e.g., `.com`). Emails lacking proper structure are correctly identified as invalid.

Scenario 8: Normalizing and Standardizing Data

Problem: Rescale numerical features into a normalized range of 0 to 1 using Min-Max scaling.

PYTHON SOLUTION

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data = {'Value': [100, 200, 300, 400, 500]}
df = pd.DataFrame(data)

# Initialize Min-Max Scaler
scaler = MinMaxScaler()
df["Normalized"] = scaler.fit_transform(df[["Value"]])
print(df)
```

EXPECTED OUTPUT

	Value	Normalized
0	100	0.00
1	200	0.25
2	300	0.50
3	400	0.75
4	500	1.00

Explanation: `MinMaxScaler()` transforms data using the formula $x_{\text{norm}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$. This maps the minimum value (100) to 0.00 and the maximum value (500) to 1.00.

Scenario 9: Saving Cleaned Data

Problem: Export the processed and cleaned DataFrame to a CSV file for downstream storage or reporting.

PYTHON SOLUTION

```
import pandas as pd

# Export DataFrame to CSV without row index numbers
df.to_csv("cleaned_data.csv", index=False)
print("Data saved successfully.")
```

EXPECTED OUTPUT

Data saved successfully.

Explanation: `df.to_csv()` saves the cleaned dataset to disk. Setting `index=False` prevents Pandas from writing unnecessary auto-incrementing row index columns into the output CSV file.

Scenario 10: Automating Data Cleanup with a Script

Problem: Create a reusable, modular Python function to automate missing value imputation and data type conversion across datasets.

PYTHON SOLUTION

```
import pandas as pd

def clean_data(data_frame):
    # Impute missing string values with 'Unknown'
    data_frame["Name"].fillna("Unknown", inplace=True)
    # Coerce invalid/missing numeric values in 'Age'
    data_frame["Age"] = pd.to_numeric(data_frame["Age"], errors="coerce")
    return data_frame

# Test data pipeline
data = {
    "Name": ["Alice", None, "Charlie"],
    "Age": [25, "Thirty", None]
}
df = pd.DataFrame(data)

# Execute automated pipeline function
df_clean = clean_data(df)
print("Automated Pipeline Result:")
print(df_clean)
```

EXPECTED OUTPUT

```
Automated Pipeline Result:
   Name  Age
0  Alice  25.0
1 Unknown  NaN
2  Charlie  NaN
```

Explanation: Encapsulating data transformation steps inside modular Python functions enables reproducible automated ETL pipelines, allowing identical cleanup procedures to be applied across multiple data batches.